1. Stack Sort (Stack Collection)

```
import java.util.Stack;
import java.util.Scanner;

public class Q1_StackSort {
    public static void sortStack(Stack<Integer> mainStack) {
        Stack<Integer> tempStack = new Stack<>();
        while (!mainStack.isEmpty()) {
            int temp = mainStack.pop();
            while (!tempStack.isEmpty() && tempStack.peek() > temp) {
                mainStack.push(tempStack.pop());
            }
            tempStack.push(temp);
        }
        while (!tempStack.isEmpty()) {
            mainStack.push(tempStack.pop());
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack<Integer> stack = new Stack<>();
        System.out.println("=== Stack Sorting Without Extra Data Structures ===");
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        System.out.println("Enter the elements:");
        for (int i = 0; i < n; i++) {
            System.out.print("Element " + (i + 1) + ": ");
            stack.push(sc.nextInt());
        }
        System.out.println("\nOriginal Stack (top to bottom): " + stack);
        sortStack(stack);
        System.out.println("Sorted Stack (top to bottom, ascending): " + stack);
        sc.close();
    }
}
```

2.

```
[annkurrrr@archlinux Expt11]$ java Q1_StackSort
=== Stack Sorting Without Extra Data Structures ===
Enter the number of elements: 5
Enter the elements:
Element 1: 1
Element 2: 54
Element 3: 32
Element 4: 6
Element 5: 2

Original Stack (top to bottom): [1, 54, 32, 6, 2]
Sorted Stack (top to bottom, ascending): [54, 32, 6, 2, 1]
```

Order Partition (LinkedList Collection)

```
import java.util.LinkedList;
import java.util.Scanner;

public class Q2_OrderPartition {
    static class Order {
        int orderId;
        String customerName;
        boolean isDelivered;
        public Order(int orderId, String customerName, boolean isDelivered) {
            this.orderId = orderId;
            this.customerName = customerName;
            this.isDelivered = isDelivered;
        }
        @Override
        public String toString() {
            return "Order{" +
                    "orderId=" + orderId +
                    ", customerName='" + customerName + '\'' +
                    ", isDelivered=" + isDelivered +
                    '}';
        }
    }

    public static LinkedList<Order> partitionOrders(LinkedList<Order> orders) {
        LinkedList<Order> result = new LinkedList<>();

        // First pass: add all undelivered orders
        for (Order order : orders) {
            if (!order.isDelivered) {
                result.add(order);
            }
        }

        // Second pass: add all delivered orders
        for (Order order : orders) {
            if (order.isDelivered) {
                result.add(order);
            }
        }
        return result;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        LinkedList<Order> orders = new LinkedList<>();
        System.out.println("=== Order Partition by Delivery Status ===");
        System.out.print("Enter the number of orders: ");
        int n = sc.nextInt();
        sc.nextLine();
        for (int i = 0; i < n; i++) {
            System.out.println("\nOrder " + (i + 1) + ":");
            System.out.print("  Order ID: ");
            int orderId = sc.nextInt();
            sc.nextLine();
```

```
            System.out.print("  Customer Name: ");
            String customerName = sc.nextLine();
            System.out.print("  Is Delivered (true/false): ");
            boolean isDelivered = sc.nextBoolean();
            sc.nextLine();
            orders.add(new Order(orderId, customerName, isDelivered));
        }
        System.out.println("\n--- Original List ---");
        for (Order order : orders) {
            System.out.println(order);
        }
        LinkedList<Order> partitionedOrders = partitionOrders(orders);
        System.out.println("\n--- Partitioned List (Undelivered before Delivered) ---");
        for (Order order : partitionedOrders) {
            System.out.println(order);
        }
        sc.close();
    }
}
```

```
[annkurrrr@archlinux Expt11]$ java Q2_OrderPartition
=== Order Partition by Delivery Status ===
Enter the number of orders: 2

Order 1:
  Order ID: 123
  Customer Name: Ankur Kunde
  Is Delivered (true/false): true

Order 2:
  Order ID: 124
  Customer Name: Ben Dover
  Is Delivered (true/false): false

--- Original List ---
Order{orderId=123, customerName='Ankur Kunde', isDelivered=true}
Order{orderId=124, customerName='Ben Dover', isDelivered=false}

--- Partitioned List (Undelivered before Delivered) ---
Order{orderId=124, customerName='Ben Dover', isDelivered=false}
Order{orderId=123, customerName='Ankur Kunde', isDelivered=true}
```

3. Contact Grouping (ArrayList + HashSet)

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.HashSet;
import java.util.Set;

public class Q3_ContactGrouping {
    static class Contact {
```

```
    String name;
    String phoneNumber;
    public Contact(String name, String phoneNumber) {
        this.name = name;
        this.phoneNumber = phoneNumber;
    }
    @Override
    public String toString() {
        return "Contact{" +
                "name='" + name + '\'' +
                ", phoneNumber='" + phoneNumber + '\'' +
                '}';
    }
}

public static void groupContactsByFirstLetter(ArrayList<Contact> contacts) {
    // Get all unique first letters
    Set<Character> letters = new HashSet<>();
    for (Contact contact : contacts) {
        if (contact.name.length() > 0) {
            letters.add(Character.toUpperCase(contact.name.charAt(0)));
        }
    }

    // Sort letters for better display
    ArrayList<Character> sortedLetters = new ArrayList<>(letters);
    sortedLetters.sort(Character::compareTo);

    // Print contacts grouped by first letter
    for (Character letter : sortedLetters) {
        System.out.println("\n--- " + letter + " ---");
        boolean found = false;
        for (Contact contact : contacts) {
            if (contact.name.length() > 0 &&
                Character.toUpperCase(contact.name.charAt(0)) == letter) {
                System.out.println(contact);
                found = true;
            }
        }
        if (!found) {
            System.out.println("No contacts");
        }
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    ArrayList<Contact> contacts = new ArrayList<>();
    System.out.println("=== Contact Grouping by First Letter ===");
    System.out.print("Enter the number of contacts: ");
    int n = sc.nextInt();
    sc.nextLine();
    for (int i = 0; i < n; i++) {
        System.out.println("\nContact " + (i + 1) + ":");
```

```
        System.out.print("  Name: ");
        String name = sc.nextLine();
        System.out.print("  Phone Number: ");
        String phoneNumber = sc.nextLine();
        contacts.add(new Contact(name, phoneNumber));
    }
    System.out.println("\n=== Contacts Grouped by First Letter ===");
    groupContactsByFirstLetter(contacts);
    sc.close();
  }
}
```

```
[annkurrrr@archlinux Expt11]$ java Q3_ContactGrouping
=== Contact Grouping by First Letter ===
Enter the number of contacts: 2

Contact 1:
  Name: Ankur Kunde
  Phone Number: 8564457654

Contact 2:
  Name: Ben Dover
  Phone Number: 7633443532

=== Contacts Grouped by First Letter ===

--- A ---
Contact{name='Ankur Kunde', phoneNumber='8564457654'}

--- B ---
Contact{name='Ben Dover', phoneNumber='7633443532'}
```

4. Vector Pair Sum (Vector Collection)

```
import java.util.Vector;
import java.util.Scanner;
import java.util.HashSet;
import java.util.Set;

public class Q4_VectorPairSum {
    public static void findPairsWithSum(Vector<Integer> vec, int targetSum) {
        Set<String> uniquePairs = new HashSet<>();
        int pairCount = 0;
        System.out.println("\nPairs found (indices i, j where vec[i] + vec[j] = " + targetSum + "):");
        for (int i = 0; i < vec.size(); i++) {
            for (int j = i + 1; j < vec.size(); j++) {
                if (vec.get(i) + vec.get(j) == targetSum) {
                    String pairKey = vec.get(i) + "+" + vec.get(j);
```

```
            if (!uniquePairs.contains(pairKey)) {
                uniquePairs.add(pairKey);
                System.out.println("  Indices (" + i + ", " + j + "): " +
                        vec.get(i) + " + " + vec.get(j) + " = " + targetSum);
                pairCount++;
            }
        }
    }
  }
  if (pairCount == 0) {
    System.out.println("  No pairs found with sum = " + targetSum);
  } else {
    System.out.println("\nTotal pairs found: " + pairCount);
  }
}

public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);
  Vector<Integer> vec = new Vector<>();
  System.out.println("Vector: Find All Pairs with Given Sum");
  System.out.print("Enter the number of elements: ");
  int n = sc.nextInt();
  System.out.println("Enter the elements:");
  for (int i = 0; i < n; i++) {
    System.out.print("Element " + (i) + ": ");
    vec.add(sc.nextInt());
  }
  System.out.println("\nVector elements: " + vec);
  System.out.print("\nEnter the target sum: ");
  int targetSum = sc.nextInt();
  findPairsWithSum(vec, targetSum);
```

```
[annkurrrr@archlinux Expt11]$ java Q4_VectorPairSum
Vector: Find All Pairs with Given Sum
Enter the number of elements: 5
Enter the elements:
Element 0: 3
Element 1: 63
Element 2: 2
Element 3: 54
Element 4: 732

Vector elements: [3, 63, 2, 54, 732]

Enter the target sum: 65

Pairs found (indices i, j where vec[i] + vec[j] = 65):
  Indices (1, 2): 63 + 2 = 65

Total pairs found: 1
```

```
    sc.close();
  }
}
```